

1. Introdução

Neste laboratório exploraremos, de forma **construtiva**, uma implementação didática do **algoritmo Simplex** em **Python**. Partiremos exatamente do espírito apresentado na última aula: acompanhar o tableau *iteração por iteração*, observando como a escolha da variável que entra e da variável que sai da base modifica a solução corrente. Ao final, as questões propõem pequenas extensões e investigações sobre os limites dessa implementação.

2. Introdução ao Algoritmo Simplex

Um problema de **PL** pede que se maximize ou minimize uma *função objetivo* linear $z = c_1x_1 + \dots + c_nx_n$ sobre n *variáveis de decisão* $x_j \geq 0$, sujeita a um conjunto de restrições lineares (igualdades ou desigualdades), cujos lados direitos (RHS - do inglês, Right Hand Side) compõem o vetor b com entradas b_i . O método **Simplex** é um procedimento **iterativo** para resolvê-lo: explora as arestas do poliedro (a região viável) saltando de vértice em vértice em direção a soluções cada vez melhores, até que nenhuma melhoria seja possível.

Neste laboratório, porém, não tentaremos construir uma função “universal”, mas algo básico e funcional. O foco será o caso construtivo estudado em aula: problemas de maximização já escritos em forma adequada ao tableau, com uma solução básica viável inicial evidente pelas variáveis de folga. As questões finais investigam o que acontece quando algumas decisões do algoritmo são alteradas.

O processo se divide em três etapas:

- **Etapa inicial – Solução Básica Viável (SBV) inicial.** Uma *base* é um conjunto de m variáveis (chamadas *básicas*) cujas colunas formam uma submatriz invertível; as demais $(n - m)$ são *não básicas* e ficam em zero. A *solução básica viável* é o vértice associado a uma base que satisfaz todas as restrições. Identificar a SBV inicial é dar o ponto de partida do algoritmo.
- **Etapa iterativa – melhoria da solução corrente.** A cada passo, escolhe-se uma direção de melhoria e realiza-se uma *troca de base* (uma variável não básica entra, uma básica sai), deslocando-se para um vértice adjacente que aumenta (ou reduz) z .
- **Etapa de parada – encerramento na otimalidade.** Quando nenhuma direção adicional melhora z , a solução corrente é declarada ótima.

2.1 O Tableau Simplex

O **Tableau Simplex** é uma representação compacta do sistema de equações lineares do problema (incluindo a função objetivo e todas as restrições) sobre a qual se aplicam operações elementares de linha (eliminação gaussiana) até atingir a solução ótima. Suas entradas seguem a notação padrão: c_j é o coeficiente da variável x_j na linha z (reflete o custo/lucro reduzido, na convenção de sinais adotada); a_{ij} é o coeficiente da variável x_j na i -ésima linha de restrição; e b_i é o RHS dessa restrição.

Para o problema *Reddy Mikks* na forma padrão (Exemplo 3.3-1 do Taha):

$$\begin{aligned}
 \max \quad & z = 5x_1 + 4x_2 + 0s_1 + 0s_2 + 0s_3 + 0s_4 \\
 \text{s.a.} \quad & 6x_1 + 4x_2 + s_1 = 24 \quad (\text{matéria-prima } M_1), \\
 & x_1 + 2x_2 + s_2 = 6 \quad (\text{matéria-prima } M_2), \\
 & -x_1 + x_2 + s_3 = 1 \quad (\text{limite de mercado}), \\
 & x_2 + s_4 = 2 \quad (\text{limite da demanda}), \\
 & x_1, x_2, s_1, \dots, s_4 \geq 0.
 \end{aligned}$$

o **Tableau Simplex** inicial é:

Base	z	x_1	x_2	s_1	s_2	s_3	s_4	Solução	
z	1	-5	-4	0	0	0	0	0	linha z
s_1	0	6	4	1	0	0	0	24	linha s_1
s_2	0	1	2	0	1	0	0	6	linha s_2
s_3	0	-1	1	0	0	1	0	1	linha s_3
s_4	0	0	1	0	0	0	1	2	linha s_4

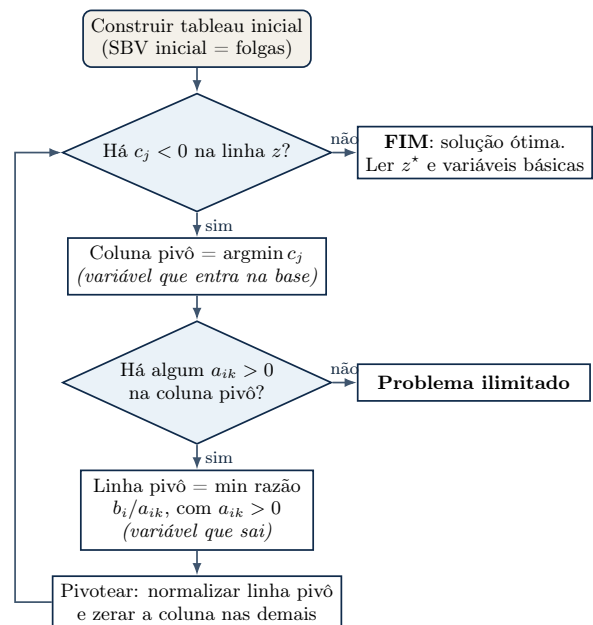
★ Dica

No livro do **Taha** (graduação, engenharias), o tableau não é deduzido formalmente; ele é apresentado como uma “fotografia” do estado atual do problema: cada **linha** é uma equação (a 1ª é z , as demais são as restrições); cada **coluna** é uma variável (de decisão x_j , de folga s_i , ou artificial); a **última coluna** guarda os b_i (RHS).

2.2 O algoritmo básico da última aula em um diagrama

Quando o problema está em Forma Padrão (vide slides) com igualdades e possui uma base inicial evidente, é fácil identificar uma SBV inicial composta pelas variáveis de folga (matriz identidade no tableau).

A partir daí, aplicamos o Simplex como uma sequência de *operações elementares de linha*, atualizando iterativamente o tableau até atingir a solução ótima. Reveja os slides da aula. O diagrama ao lado resume o fluxo completo do algoritmo implementado neste laboratório. Cada decisão (losango) corresponde a um teste na linha z ou na coluna pivô; cada processo (retângulo) corresponde a uma operação sobre o tableau. O laço de retorno representa uma iteração do algoritmo.



3. Estudo e Compreensão do Algoritmo Simplex

Nesta seção, implementamos um Simplex básico em Python. Os parâmetros do código refletem diretamente a notação acima: **matriz_A** contém os coeficientes a_{ij} das restrições, **vetor_b** contém os RHS b_i , e **vetor_c** contém os coeficientes da função objetivo na convenção do tableau adotado neste laboratório, isto é, com sinal trocado em relação aos coeficientes originais da maximização.

Atenção

A implementação abaixo segue o caso construtivo estudado em aula: problemas de maximização já escritos na forma de tableau, com uma SBV inicial evidente pelas variáveis de folga. Nas questões finais, você investigará o que muda quando algumas dessas escolhas são alteradas.

Passo 1 – Ambiente no Colab

- (1) Abra o [Google Colab](#);
- (2) Crie um novo Notebook.

Passo 2 – Implementação da função Simplex

Na primeira célula, escreva o código abaixo e rode o exemplo de uso:

```

import numpy as np

def simplex(matriz_A, vetor_b, vetor_c):
    linhas, colunas = matriz_A.shape
    # Cria a tabela de floats (importante para divisões fracionárias)
    tabela = np.zeros((linhas + 1, colunas + 1), dtype=float)

    # Preenche o tableau
    tabela[1:linhas + 1, 0:colunas] = matriz_A
    tabela[0, 0:colunas] = vetor_c
    tabela[1:linhas + 1, -1] = vetor_b

    print('*****')
    print('Tableau Inicial')
    print(tabela)

    iteracao = 1
    while True:
        # Condição de parada: não há coeficientes negativos na linha Z (linha 0)
        if not any(tabela[0, 0:colunas] < 0):

```

```

print('Nao ha mais coef. negativos.')
print('***** FIM *****')
break

# Escolhe a coluna pivô (a mais negativa)
coluna_pivo = np.argmin(tabela[0, 0:colunas])
linha_pivo = -1
menor_razao = float('inf')

# Teste da razão mínima para escolher a linha pivô
for i in range(1, linhas + 1):
    if tabela[i, coluna_pivo] > 0:
        razao = tabela[i, -1] / tabela[i, coluna_pivo]
        if razao < menor_razao:
            menor_razao = razao
            linha_pivo = i

# Verifica se o problema é ilimitado
if linha_pivo == -1:
    print("Problema ilimitado (sem linha pivo valida).")
    break

print('*****')
print(f"Linha pivo = {linha_pivo}")
print('*****')

# Operações de pivoteamento (Gauss-Jordan)
tabela[linha_pivo, :] /= tabela[linha_pivo, coluna_pivo]
for i in range(linhas + 1):
    if i != linha_pivo:
        tabela[i, :] -= tabela[i, coluna_pivo] * tabela[linha_pivo, :]

print(f"Tableau depois da iteracao [{iteracao}]")
print(np.round(tabela, 2)) # Arredondado para ficar mais legível
iteracao += 1

return tabela

# Exemplo de uso para Q1
matriz_A = np.array([
    [1, 0, 1, 0, 0],
    [0, 1, 0, 1, 0],
    [1, 1, 0, 0, 1]
])
vetor_b = np.array([4, 6, 8])
vetor_c = np.array([-2, -5, 0, 0, 0])

resultado = simplex(matriz_A, vetor_b, vetor_c)

print('\nResultado Final:')
print(np.round(resultado, 2))

```

Atenção

Os coeficientes da função objetivo são passados *já com o sinal trocado* (i.e., $\text{vetor_c} = [-2, -5, 0, 0, 0]$ para $\max z = 2x_1 + 5x_2$). Isso acontece porque a linha da função objetivo é escrita como $z - 2x_1 - 5x_2 = 0$, e o algoritmo busca eliminar coeficientes negativos da linha z . Se você trocar o sentido do problema (*minimização*), repense essa convenção de sinais!

★ Por que só $a_{ik} > 0$ na razão mínima?

No código, a razão b_i/a_{ik} só é calculada se $\text{tabela}[i, \text{coluna_pivo}] > 0$. Por quê?

- **Geometria:** se a região viável for “aberta” na direção de crescimento de z , o lucro cresce para sempre ($z \rightarrow \infty$).
- **Álgebra:** $a_{ik} > 0$ indica *consumo* de recurso. Se todos os coeficientes da coluna pivô forem ≤ 0 , a variável pode crescer ao infinito sem esgotar nenhum recurso — e o código sinaliza **Ilimitado**.

Não se preocupe, neste momento, em construir uma função “universal”. A ideia é observar o mecanismo do Simplex no caso construtivo estudado em aula. As questões seguintes exploram o que acontece quando mudamos algumas decisões do algoritmo.

Passo 3 – Práticas no seu Notebook Colab

Faça as práticas das Questões abaixo. Use células de **Markdown** para explicar e ilustrar tal como num relatório.

4. Questões

Q1. Resolução via Simplex Básico

Seja o problema de maximização com restrições do tipo “menor ou igual”:

$$\max z = 2x_1 + 5x_2$$

$$\text{s.a. } x_1 \leq 4, x_2 \leq 6, x_1 + x_2 \leq 8, x_i \geq 0.$$

Convertendo para a forma com igualdades, adicionando variáveis de folga s_j :

$$\max z - 2x_1 - 5x_2 + 0s_1 + 0s_2 + 0s_3 = 0$$

$$x_1 + s_1 = 4$$

$$x_2 + s_2 = 6 \quad x_i, s_j \geq 0.$$

$$x_1 + x_2 + s_3 = 8$$

- Use a função `simplex` para resolver. Defina as matrizes, invoque a função e observe os tableaux.
- Localize as **variáveis básicas** e **não básicas** no tableau final. Explique como você as identificou a partir das colunas do tableau.
- Identifique explicitamente x_1^* , x_2^* , s_1^* , s_2^* , s_3^* e o valor final z^* .
- Identifique quais restrições estão ativas e quais possuem folga.
- Resolva também pelo **método gráfico** e compare com o resultado do Simplex.

Q2. Comparação Exemplo Reddy Mikks

Leia a Seção 3.3.2 do Taha (p. 43). Compreenda cada iteração do Exemplo 3.3-1 e compare os tableaux do livro com os produzidos pelo algoritmo. Observe especialmente a escolha da coluna pivô, da linha pivô e a ordem em que as variáveis entram na base. *Aconteceu algo errado?* Explique.

Q3. Variante: coeficiente menos negativo

E se, em vez de escolher o coeficiente mais negativo da linha z , você escolhesse o coeficiente **negativo mais próximo de zero** para entrar na base? Por exemplo, entre -5 e -4 , o coeficiente menos negativo é -4 . Modifique o algoritmo e mostre o que acontece, apresentando **todos os tableaux**.

Compare o número de iterações obtido com a regra do mais negativo e com a regra do menos negativo. Compare também, quando possível, o caminho percorrido no espaço das soluções.

★ Dica

A *regra do mais negativo* (regra de Dantzig) privilegia a maior taxa de melhoria. Trocá-la **não compromete a otimalidade**, mas pode aumentar o número de iterações ou alterar o caminho percorrido até a solução ótima.

Q4. Generalização: max ou min

Leia o **Resumo do Simplex** (Taha, p. 45). Reveja o fluxograma acima. Explique a **condição de otimalidade** no tableau. *Em que ela difere para maximização e minimização, segundo a convenção de sinais adotada?*

Modifique o algoritmo e o fluxograma para aceitar **maximização ou minimização** usando um parâmetro, por exemplo `sentido='max'` ou `sentido='min'`. Você pode implementar a minimização convertendo-a em uma maximização equivalente, desde que explique claramente a convenção utilizada e como o valor ótimo da minimização é recuperado ao final. Ilustre com um exemplo.

Entrega. Submeta o *Notebook Colab* (.ipynb) com células de Markdown explicando cada etapa e saídas executadas.

Referências

H. Taha, *Pesquisa Operacional*, 8ª Ed. • Biblioteca NumPy: <https://numpy.org/> • Google Colab: <https://colab.research.google.com/>.